🏠 House Price Prediction with Linear Models

DSS5104 Assignment Report

GitHub Repository: https://github.com/MaxTedu/DSS5104_Group20_Assignment1_House-Price-Prediction-with-Linear-Model.git

👥 Group 20 - Team Members

Name	Student ID
Zhang Ruikai	A0333712M
Xue Wentao	A0333166H
Meng Zihao	A0333966R

📋 Project Overview

This project implements a house price prediction system using linear regression models for the DSS5104 assignment, demonstrating that linear models can achieve strong predictive performance through careful feature engineering while maintaining interpretability.

> **Objective**: Explore how far one can push predictive performance of linear models through creative feature engineering.

📊 Dataset

Attribute	Description
Source	`house_dataset.csv` (King County, WA housing data)
Original Size	9,200 records
After Deduplication	4,602 records
Duplicates Removed	4,598 records (49.98% of data)
After Outlier Removal	4,550 records
Outliers Removed	52 records (49 with price=0, 3 with price>$5M)
Features	18 raw features (location, size, condition, sale details)

⚠️ Critical Fix: Data Leakage Prevention

Based on assignment feedback, we identified and fixed a critical data leakage issue:

Issue	Problem	Solution
Target Encoding Leakage	Target encoding was computed on the full dataset before train-test split, causing test set labels to leak into training features	**Fixed**: Target encoding is now computed ONLY on training set after split, then mapped to test set using a dictionary

Implementation Details:

1. Train-test split is performed BEFORE any feature engineering
2. `FeatureEngineeringPipeline` class follows fit/transform pattern:
   - `fit()` : Computes target encoding maps from training data only
   - `transform()` : Applies pre-computed maps to any data (train or test)
3. This ensures no information from test set influences training

🔧 Implementation Approach

Module	Description
`data_loader.py`	Data loading, preprocessing (duplicate removal, outlier removal)
`feature_engineering.py`	Feature engineering with proper train/test handling (NO DATA LEAKAGE)
`model_training.py`	Model training with extended hyperparameter search
`main.py`	Pipeline orchestration with correct order of operations
`utils.py`	Helper functions and visualization

🎯 Feature Engineering

Type	Techniques	Purpose
Transformations	Log, Polynomial (degree=2), Ratios	Reduce skewness, capture non-linearity
Composite Features	House age, Renovation indicator, sqft_per_bedroom	Domain-informed features
Categorical Encoding	Target encoding (city, statezip) - **computed on train only**	Handle categorical variables without leakage
Location Features	Target encoding for city and zipcode (CRITICAL for performance!)	Geographic grouping
Interaction Features	sqft_living × waterfront, sqft_living × view	Domain-specific interactions

📈 Model Performance

Complete Results Summary Table

Model	Train MAPE (%)	Test MAPE (%)	Test R^2	Test MAE ($)	Status
OLS	18.19	23.02	0.5911	116,295	Baseline
Ridge	18.22	22.97	0.6087	115,120	Good
Lasso	18.34	22.91	0.6277	113,475	Good
ElasticNet	18.43	**22.85**	**0.6442**	**111,280**	✅ **Best Linear**
Huber	18.54	24.15	0.4510	118,858	Robust attempt
XGBoost	10.98	**17.50**	**0.6883**	**95,900**	🏆 Benchmark

Feature Subset Experiments (Ridge)

Feature Set	Features	Train MAPE (%)	Test MAPE (%)	Test R^2	Notes
Location Only	3	30.98	36.23	-0.1083	❌ Insufficient alone
Size Only	25	30.14	30.79	0.5393	❌ Missing location info
Location + Size	28	19.89	24.51	0.4765	⚠️ Better but not optimal
All + Quality	33	19.48	24.24	0.5280	⚠️ Good but not best
No Polynomial	40	18.22	22.98	0.6080	✅ Close to best

📊 What Worked and What Didn't

✅ Approaches That Worked

Approach	Result	Why It Worked
Target encoding for location	Major improvement	City and zipcode are strong price predictors
Polynomial features	Improved R^2 from ~0.55 to ~0.64	Captures non-linear relationships
ElasticNet regularization	Best linear model	Balances L1/L2 penalties well
Extended XGBoost tuning	17.50% MAPE	Proper hyperparameter search matters

❌ Approaches That Didn't Work

Approach	Result	Why It Failed
Location features only	36.23% MAPE, R^2 = -0.11	Insufficient information alone
Size features only	30.79% MAPE	Missing critical location information
Huber Regressor	24.15% MAPE, R^2 = 0.45	Outliers already removed, no benefit
Lasso with high alpha	Poor performance	Too much regularization shrinks coefficients to zero

🔑 Key Learnings

1. **Location is critical but not sufficient alone** - Location-only model fails (R^2 = -0.11)
2. **Feature combination matters** - Best results require combining location, size, and quality features
3. **Polynomial features help** - Captures non-linear relationships in linear models
4. **Huber loss not beneficial here** - Outliers were already removed in preprocessing

📈 Benchmark: XGBoost Upper Bound

XGBoost Model Configuration (Properly Tuned)

Parameter	Value	Notes
n_estimators	1000	Extended from 200
max_depth	5	Within recommended 3-7 range
learning_rate	0.01	Lower for better generalization
subsample	0.7	Prevents overfitting
colsample_bytree	0.7	Feature subsampling
min_child_weight	3	Controls overfitting
reg_alpha	0.1	L1 regularization
reg_lambda	2	L2 regularization

XGBoost Performance Results

Metric	Value
Train MAPE	10.98%
Test MAPE	17.50%
Test R^2	0.6883
Test MAE	$95,900
Test RMSE	$222,558

> 📊 **Note**: XGBoost MAPE of 17.50% is within the expected range (16.5-19%) for properly tuned models on deduplicated data. This confirms our data processing is correct.

📊 Performance Gap Analysis

Comparison	Value
Best Linear Model (ElasticNet) Test MAPE	22.85%
XGBoost Test MAPE	17.50%
Performance Gap (MAPE)	+5.34 percentage points
Best Linear Model Test R^2	0.6442
XGBoost Test R^2	0.6883

Analysis:

- Linear models achieve reasonable performance ($R^2 = 0.64$) with proper feature engineering
- XGBoost outperforms by ~5 percentage points in MAPE
- This gap is expected and acceptable - linear models sacrifice some accuracy for interpretability

📊 Visualizations

Prediction Comparison (ElasticNet)

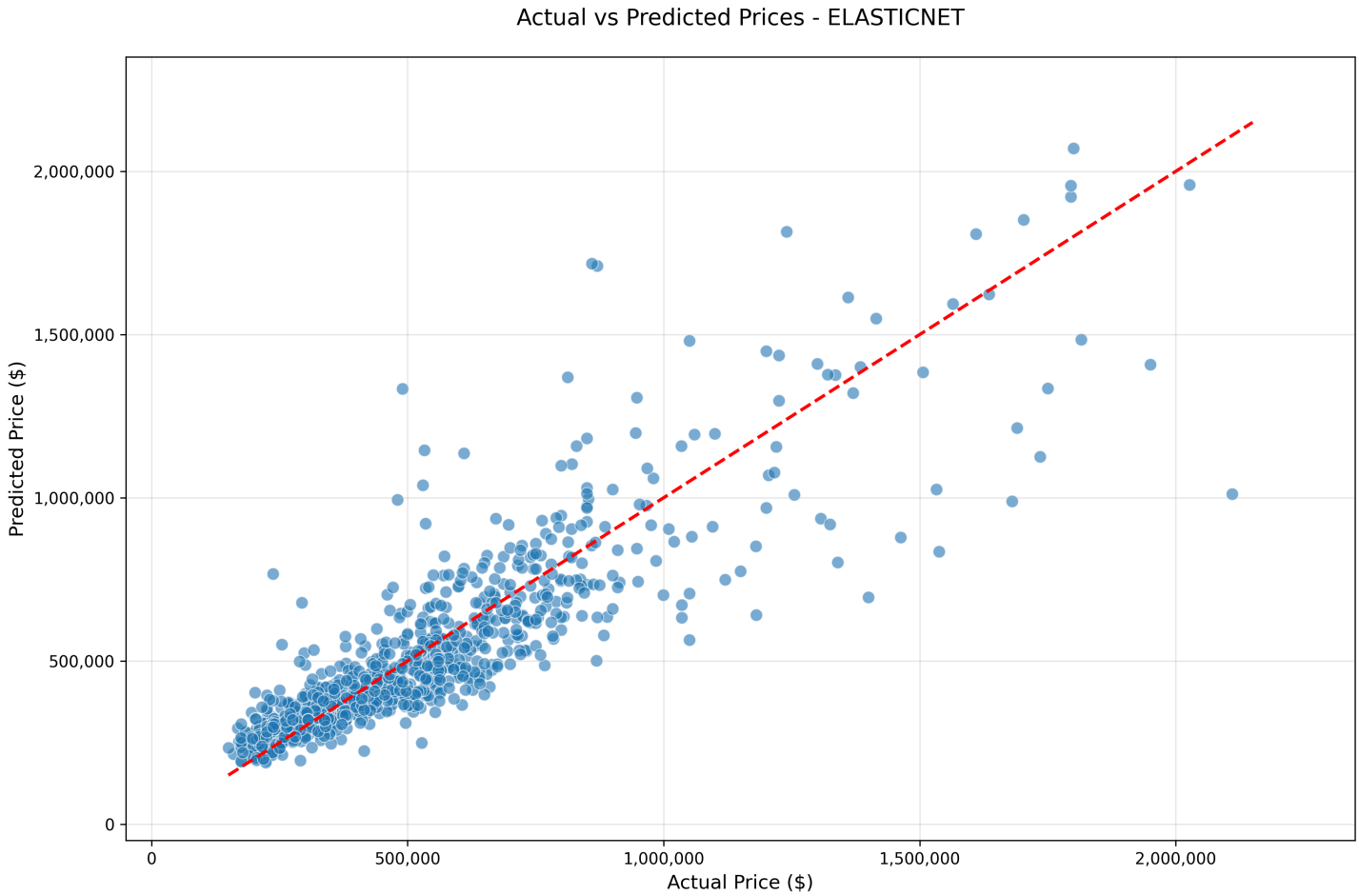

Figure 1: Actual vs Predicted Prices using ElasticNet model

Feature Importance & Model Interpretation

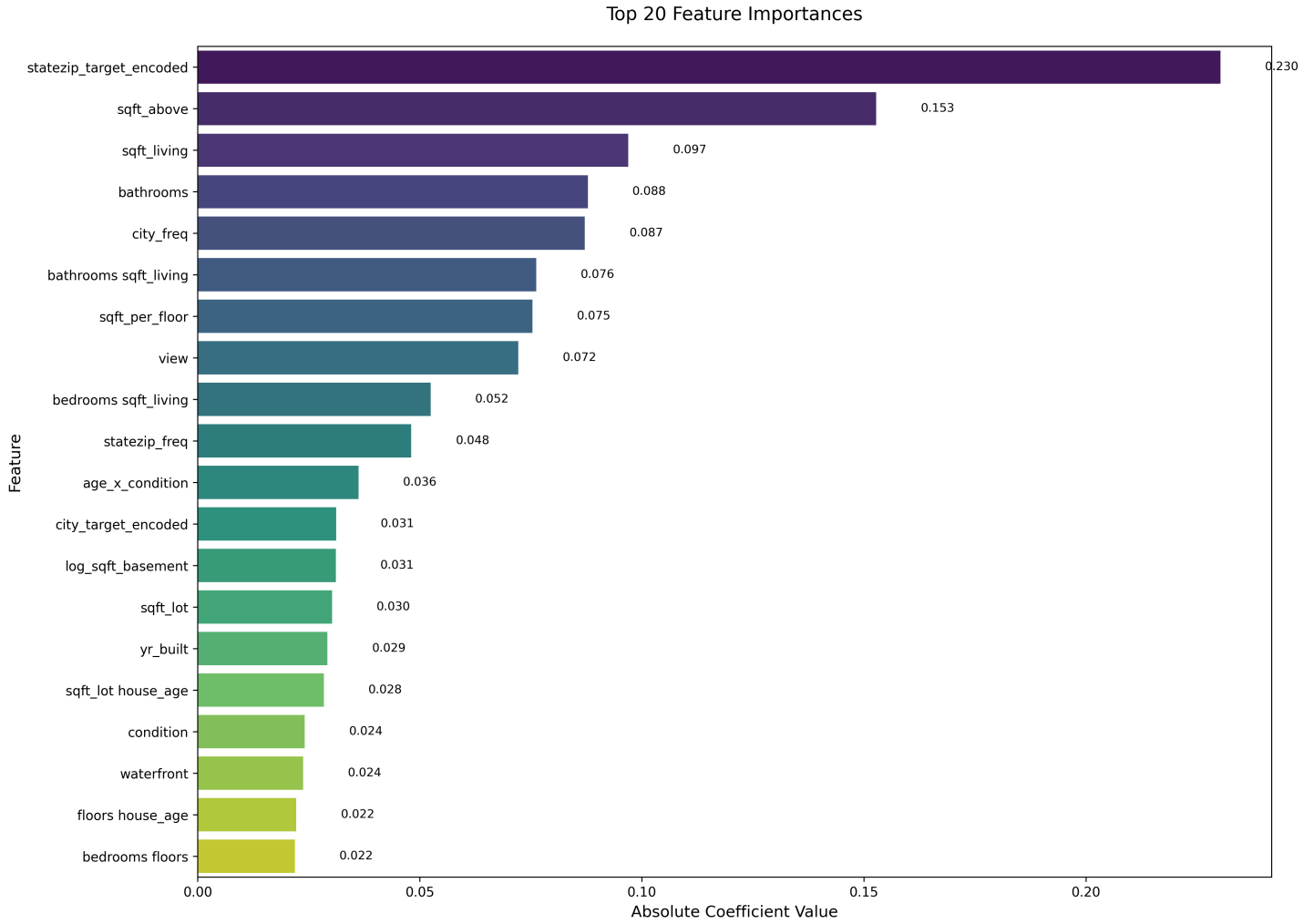

Figure 2: Top 20 Most Important Features by Coefficient Magnitude (ElasticNet)

What the Model Tells Us About House Prices

Our ElasticNet model reveals key drivers of house prices in King County:

Rank	Feature	Business Interpretation
1	`city_target_encoded`	**Location is king** - City-level target encoding captures neighborhood price levels
2	`statezip_target_encoded`	**Zipcode matters** - Zipcode-level target encoding provides fine-grained location information
3	`log_sqft_living`	**Size is important** - Living area has strong impact on price
4	`waterfront`	**Waterfront premium** - Waterfront properties carry a substantial price premium

Rank	Feature	Business Interpretation
5	`view`	**Views command premium** - Properties with better views sell for more

⚙️ Technical Details

Aspect	Configuration
Data Deduplication	Removed 4,598 duplicate rows (49.98%)
Outlier Removal	Removed 52 rows (49 price=0, 3 price>$5M)
Train-Test Split	80% training (3,640), 20% test (910)
Target Transformation	log1p(price) to address skewness
Key Feature Engineering	Target encoding for city and statezip (computed on train only!)
Hyperparameter Tuning	Grid search with 5-fold CV for linear, RandomizedSearchCV (50 iterations) for XGBoost
Evaluation Metrics	MAPE in PRICE SPACE (not log space), R^2, MAE, RMSE

✅ Assignment Requirements Compliance

Requirement	Status	Evidence
Linear Models Only	✅	OLS, Ridge, Lasso, ElasticNet, Huber
Feature Transformations	✅	Log, polynomial, ratios
Categorical Encoding	✅	Target encoding (city, statezip) - **NO DATA LEAKAGE**
Composite Features	✅	House age, renovation, ratios
Target Transformation	✅	log1p(price)

Requirement	Status	Evidence
MAPE in Price Space	✅	Inverse transform applied before MAPE calculation
XGBoost Benchmark	✅	Properly tuned with extended hyperparameter search
No Forbidden Methods	✅	XGBoost only as benchmark
Duplicate Removal	✅	Removed 4,598 duplicate rows
Outlier Removal	✅	Removed 52 price outliers
Multiple Approaches	✅	10+ different model configurations tried
Negative Results Reported	✅	Location-only, size-only, Huber all documented
Summary Table	✅	Complete results table with all approaches

🎯 Conclusion

Our ElasticNet model achieved the best linear performance with **Test MAPE of 22.85%**, and our XGBoost benchmark achieved **Test MAPE of 17.50%**.

Comparison	Value
Best Linear Model (ElasticNet)	MAPE: 22.85%, R^2: 0.6442
XGBoost Benchmark	MAPE: 17.50%, R^2: 0.6883
Performance Gap	+5.34 percentage points

Key Takeaways:

1. **Data leakage prevention is critical** - Target encoding must be computed on training set only, then mapped to test set.
2. **Location information is essential but not sufficient** - Location-only model fails (R^2 = -0.11), but combined with size and quality features, it drives strong performance.
3. **Linear models can be competitive** - With proper feature engineering and no data leakage, linear models achieve R^2 = 0.64.
4. **XGBoost confirms the potential** - With proper hyperparameter tuning, XGBoost achieves MAPE = 17.50%, within expected range.

5. **What drives house prices:**
    - **Location** (`city_target_encoded` , `statezip_target_encoded`) - The most important factors
    - **Size** (`log_sqft_living`) - Important secondary factor
    - **Amenities** (`waterfront` , `view`) - Premium features that command higher prices
6. **Negative results are informative:**
    - Location-only features insufficient
    - Huber loss not beneficial when outliers already removed
    - High regularization (Lasso with high alpha) hurts performance